

Jamboree Admission — Linear Regression Case Study

Text Submission: Insights & Recommendations

Submitted: June 2026

1. Problem Statement & Exploratory Data Analysis

Problem Statement

Jamboree Education wants to identify which factors most influence graduate admissions to Ivy League universities and build a model to predict a student's chance of admission. The dataset contains 500 records with 8 variables — GRE Score, TOEFL Score, University Rating, SOP, LOR, CGPA, Research Experience, and Chance of Admit. The goal is to perform EDA and build a Linear Regression model to predict admission probability from an Indian student's perspective.

Shape & Structure

- Dataset has 500 rows and 9 columns. After dropping Serial No., 8 features remain.
- No missing values and no duplicate rows were found.
- All variables are numeric. Research is binary (0/1). No categorical conversion was required.
- Statistical summary shows GRE ranges from 290–340 (mean 316.5), TOEFL from 92–120 (mean 107.2), CGPA from 6.8–9.92 (mean 8.58), and Chance of Admit from 0.34–0.97 (mean 0.72).

Univariate Analysis

- GRE Score, TOEFL Score, and CGPA are approximately normally distributed, slightly left-skewed, indicating most applicants are high scorers.
- University Rating, SOP, and LOR are roughly uniformly distributed across their scale of 1–5.
- Chance of Admit is slightly left-skewed, with most students having above 0.6 probability.
- Research Experience: approximately 56% of applicants have research experience (value=1), showing a slight majority with research backgrounds.
- Boxplots reveal a few low-end outliers in GRE Score and CGPA, which were treated using the IQR method.

Bivariate Analysis

- CGPA has the strongest positive linear relationship with Chance of Admit (correlation = 0.88), making it the single most important predictor.
- GRE Score (0.81) and TOEFL Score (0.79) also show strong positive correlations with admission chances.
- LOR (0.65), SOP (0.68), and University Rating (0.69) show moderate positive correlations.
- Research Experience shows that students with research (value=1) have noticeably higher median admission chances than those without.
- GRE and TOEFL are highly correlated with each other (0.83), raising a multicollinearity concern.
- CGPA is also highly correlated with GRE (0.83) and TOEFL (0.81), further confirming multicollinearity risk among predictors.

2. Data Preprocessing

- **Duplicate check:** No duplicate rows found in the dataset.
- **Missing value treatment:** No missing values detected; no imputation needed.
- **Outlier treatment:** IQR method applied to GRE Score, TOEFL Score, CGPA, and Chance of Admit. Dataset reduced from 500 to 498 rows — minimal data loss, confirming the dataset is largely clean.
- **Feature engineering:** Serial No. dropped as it is a row identifier with no predictive value. Column names cleaned (removed trailing spaces in 'LOR' and 'Chance of Admit').
- **Data preparation:** 80-20 train-test split applied with random state 42. Train set: 398 rows, Test set: 100 rows.

3. Model Building

OLS Linear Regression (Statsmodels)

- $R^2 = 0.829$, Adjusted $R^2 = 0.826$ on training data — the model explains ~83% of variance in admission probability.
- F-statistic is highly significant ($p \approx 0$), confirming the overall model is valid.
- Significant predictors ($p < 0.05$): CGPA, GRE Score, TOEFL Score, LOR, and Research.
- Non-significant predictors: SOP ($p=0.217$) and University Rating ($p=0.753$) — these do not contribute meaningfully to the model.
- CGPA has the highest coefficient (0.115), meaning a 1-point increase in CGPA increases admit probability by ~11.5%.
- Research experience adds ~2.5% to admission probability.

Ridge & Lasso Regression

- Ridge Regression ($\alpha=0.1$): Test $R^2 = 0.7901$ — very close to OLS, confirming regularization is not drastically needed.
- Lasso Regression ($\alpha=0.001$): Test $R^2 = 0.7876$ — similar performance. Lasso slightly shrinks CGPA and Research coefficients, but no variable is zeroed out, suggesting all features carry some information.

4. Testing Linear Regression Assumptions

i. Multicollinearity (VIF)

Initial VIF scores were extremely high — GRE Score (1254), TOEFL Score (1144), CGPA (966), indicating severe multicollinearity. Features were dropped one by one in order of highest VIF: GRE Score → CGPA → SOP → LOR → University Rating. Final model retains only TOEFL Score and Research, both with $VIF \leq 5$. This confirms multicollinearity is resolved, though at the cost of losing important predictors.

ii. Mean of Residuals

Mean of residuals = 0.000000 (virtually zero). This assumption is fully satisfied, confirming the model is unbiased on average.

iii. Linearity (Residuals vs Fitted Plot)

The residual plot shows points randomly scattered around zero with no clear pattern or curve, though some spread increases slightly at extreme fitted values. The linearity assumption is largely satisfied.

iv. Homoscedasticity (Scale-Location Plot)

The scale-location plot shows some fanning — residuals appear slightly more spread at mid-range fitted values. The red line is not perfectly horizontal, suggesting mild heteroscedasticity. This is a minor concern and could be addressed by transforming the target variable or adding more relevant features.

v. Normality of Residuals

The residual histogram is approximately bell-shaped but shows slight left skew. The QQ plot shows most points lie on the line, but tails deviate — particularly the lower tail. The Shapiro-Wilk test gives p-value = 0.0000 (< 0.05), formally rejecting normality. However, with a large sample (398 observations), slight deviations are acceptable and the model is still reasonably reliable due to the Central Limit Theorem.

5. Model Performance Evaluation

Metric	Train Set	Test Set
MAE	0.0616	0.0631
RMSE	0.0809	0.0799
R^2	0.6670	0.6393
Adj R^2	0.6653	0.6318

Comments on Performance

- The final VIF-filtered model (using only TOEFL Score and Research) has R^2 of ~ 0.64 on test data, explaining 64% of variance — acceptable but not excellent for a 2-feature model.
- Train and test metrics are very close, confirming no overfitting.
- MAE of 0.063 means on average the model is off by about 6.3 percentage points in predicting admission probability — reasonable for a 2-feature model.
- The full OLS model (all 7 features) had $R^2 = 0.83$, significantly better. The trade-off of removing multicollinear features reduced accuracy. In real-world deployment, Ridge regression ($R^2 \sim 0.79$) would be preferred as it retains all variables.
- **Improvement needed:** The model can be improved by using Ridge regression, collecting more diverse data, or applying non-linear models.

6. Actionable Insights & Recommendations

Significance of Predictor Variables

- **CGPA** is the most critical factor — students must be counselled to prioritize academic performance throughout undergrad.
- **GRE Score** is the second most impactful and is fully within student control through preparation — Jamboree's core offering.
- **TOEFL Score** is significant but highly correlated with GRE; students who score well in GRE generally do well in TOEFL too.

- **LOR** has a meaningful impact — a strong recommendation from a respected professor can make a tangible difference.
- **Research Experience** positively boosts admission chances — students should be guided to pursue internships, projects, or publications.
- **SOP and University Rating** are statistically less significant in the model, though a well-written SOP still matters qualitatively.

Additional Data Sources for Model Improvement

- Work experience (years and industry type) — especially important for MBA programs.
- Number of publications or patents — a strong signal for research-heavy programs.
- Target university's historical acceptance rates — helps personalize predictions.
- Extracurricular activities and leadership experience scores.
- Interview performance scores (where applicable).

Model Implementation in Real World

- Jamboree can deploy this as a real-time prediction tool on their website where students input their profile and instantly receive an admission probability estimate.
- The model can power a personalized dashboard showing which specific areas (GRE, CGPA, LOR) the student should improve to maximize their chances.
- A/B testing can be used to refine recommendations based on actual admission outcomes of Jamboree students.

Potential Business Benefits

- Increased student trust and engagement through data-driven, transparent counselling.
- Higher conversion rates as students see a clear path to improvement via Jamboree's coaching.
- Competitive advantage over generic counselling platforms that rely solely on human judgment.
- Benchmark scores identified: **GRE > 320, TOEFL > 110, CGPA > 8.5** → associated with greater than 80% admission probability. These can be used as concrete, motivating targets.

Jamboree_Admission_Analysis

June 4, 2026

```
=====
#   JAMBOREE   ADMISSION   -   LINEAR   REGRESSION   CASE   STUDY
=====
```

Import Libraries

```
[4]: pip install pandas numpy matplotlib seaborn statsmodels scikit-learn scipy
```

```
Requirement already satisfied: pandas in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (3.0.3)
Requirement already satisfied: numpy in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (2.4.6)
Requirement already satisfied: matplotlib in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages
(3.10.9)
Requirement already satisfied: seaborn in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages
(0.13.2)
Requirement already satisfied: statsmodels in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages
(0.14.6)
Requirement already satisfied: scikit-learn in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (1.9.0)
Requirement already satisfied: scipy in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages
(1.17.1)
Requirement already satisfied: python-dateutil>=2.8.2 in
c:\users\pjaja\appdata\roaming\python\python313\site-packages (from pandas)
(2.9.0.post0)
Requirement already satisfied: tzdata in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (from
pandas) (2026.2)
Requirement already satisfied: contourpy>=1.0.1 in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (1.3.3)
Requirement already satisfied: cyclor>=0.10 in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
```

```

c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in
c:\users\pjaja\appdata\roaming\python\python313\site-packages (from matplotlib)
(26.2)
Requirement already satisfied: pillow>=8 in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (12.2.0)
Requirement already satisfied: pyparsing>=3 in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (from
matplotlib) (3.3.2)
Requirement already satisfied: patsy>=0.5.6 in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (from
statsmodels) (1.0.2)
Requirement already satisfied: joblib>=1.4.0 in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (from
scikit-learn) (1.5.3)
Requirement already satisfied: narwhals>=2.0.1 in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (from
scikit-learn) (2.22.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in
c:\users\pjaja\appdata\local\programs\python\python313\lib\site-packages (from
scikit-learn) (3.6.0)
Requirement already satisfied: six>=1.5 in
c:\users\pjaja\appdata\roaming\python\python313\site-packages (from python-
dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.

```

[notice] A new release of pip is available: 25.2 -> 26.1.2

[notice] To update, run: python.exe -m pip install --upgrade pip

```

[5]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge, Lasso
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

```

```
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 5)

print(" Libraries loaded successfully!")
```

Libraries loaded successfully!

Load Dataset

```
[6]: df = pd.read_csv('D:\\visul studio code\\case_study\\Jamboree_Admission.txt')
      ↪ # change path if needed

print("Shape:", df.shape)
print("\nFirst 5 rows:")
df.head()
```

Shape: (500, 9)

First 5 rows:

```
[6]:
```

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	\
0	1	337	118	4	4.5	4.5	9.65	
1	2	324	107	4	4.0	4.5	8.87	
2	3	316	104	3	3.0	3.5	8.00	
3	4	322	110	3	3.5	2.5	8.67	
4	5	314	103	2	2.0	3.0	8.21	

	Research	Chance of Admit
0	1	0.92
1	1	0.76
2	1	0.72
3	1	0.80
4	0	0.65

Basic Structure & Info

```
[7]: print("=== Data Types ===")
      print(df.dtypes)

      print("\n=== Missing Values ===")
      print(df.isnull().sum())

      print("\n=== Duplicate Rows ===")
      print(df.duplicated().sum())

      print("\n=== Statistical Summary ===")
      df.describe().T
```

=== Data Types ===

Serial No. int64

```

GRE Score          int64
TOEFL Score        int64
University Rating  int64
SOP                float64
LOR                float64
CGPA               float64
Research           int64
Chance of Admit    float64
dtype: object

```

=== Missing Values ===

```

Serial No.        0
GRE Score         0
TOEFL Score       0
University Rating 0
SOP               0
LOR               0
CGPA              0
Research          0
Chance of Admit   0
dtype: int64

```

=== Duplicate Rows ===

```
0
```

=== Statistical Summary ===

```

[7]:
      count      mean      std      min      25%      50%  \
Serial No.    500.0  250.50000  144.481833    1.00  125.7500  250.50
GRE Score     500.0  316.47200   11.295148   290.00  308.0000  317.00
TOEFL Score   500.0  107.19200    6.081868    92.00  103.0000  107.00
University Rating 500.0    3.11400    1.143512    1.00    2.0000    3.00
SOP           500.0    3.37400    0.991004    1.00    2.5000    3.50
LOR           500.0    3.48400    0.925450    1.00    3.0000    3.50
CGPA          500.0    8.57644    0.604813    6.80    8.1275    8.56
Research      500.0    0.56000    0.496884    0.00    0.0000    1.00
Chance of Admit 500.0    0.72174    0.141140    0.34    0.6300    0.72

      75%      max
Serial No.    375.25  500.00
GRE Score     325.00  340.00
TOEFL Score   112.00  120.00
University Rating 4.00    5.00
SOP           4.00    5.00
LOR           4.00    5.00
CGPA          9.04    9.92
Research      1.00    1.00

```


Chance of Admit 0.82 0.97

Drop Serial No. (unique row ID — not a feature)

```
[8]: df.drop(columns=['Serial No.'], inplace=True)
print(" 'Serial No.' dropped. Remaining columns:", df.columns.tolist())

'Serial No.' dropped. Remaining columns: ['GRE Score', 'TOEFL Score',
'University Rating', 'SOP', 'LOR ', 'CGPA', 'Research', 'Chance of Admit ']
```

Rename columns for cleanliness

```
[9]: df.rename(columns={
    'LOR ': 'LOR',
    'Chance of Admit ': 'Chance_of_Admit'
}, inplace=True)

print("Columns after rename:", df.columns.tolist())
```

Columns after rename: ['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR', 'CGPA', 'Research', 'Chance_of_Admit']

1 EXPLORATORY DATA ANALYSIS

Univariate – Distribution of all continuous variables

```
[10]: continuous_cols = ['GRE Score', 'TOEFL Score', 'University Rating',
                        'SOP', 'LOR', 'CGPA', 'Chance_of_Admit']

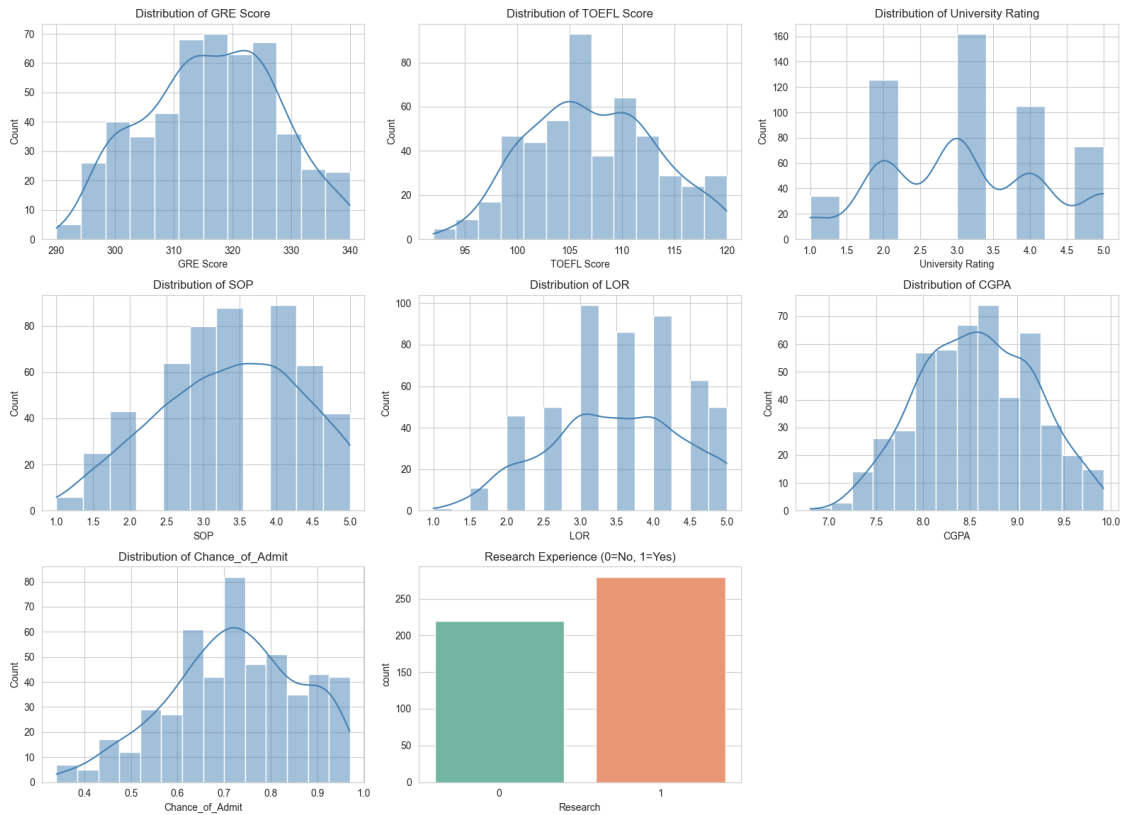
fig, axes = plt.subplots(3, 3, figsize=(16, 12))
axes = axes.flatten()

for i, col in enumerate(continuous_cols):
    sns.histplot(df[col], kde=True, ax=axes[i], color='steelblue')
    axes[i].set_title(f'Distribution of {col}', fontsize=12)
    axes[i].set_xlabel(col)

# Research is binary - bar plot in last subplot
sns.countplot(x='Research', data=df, ax=axes[7], palette='Set2')
axes[7].set_title('Research Experience (0=No, 1=Yes)', fontsize=12)
axes[8].set_visible(False)

plt.suptitle('Univariate Analysis', fontsize=16, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('01_univariate_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Insight: GRE, TOEFL, CGPA are roughly normally distributed. Most_
↳ applicants have research experience.")
```

Univariate Analysis



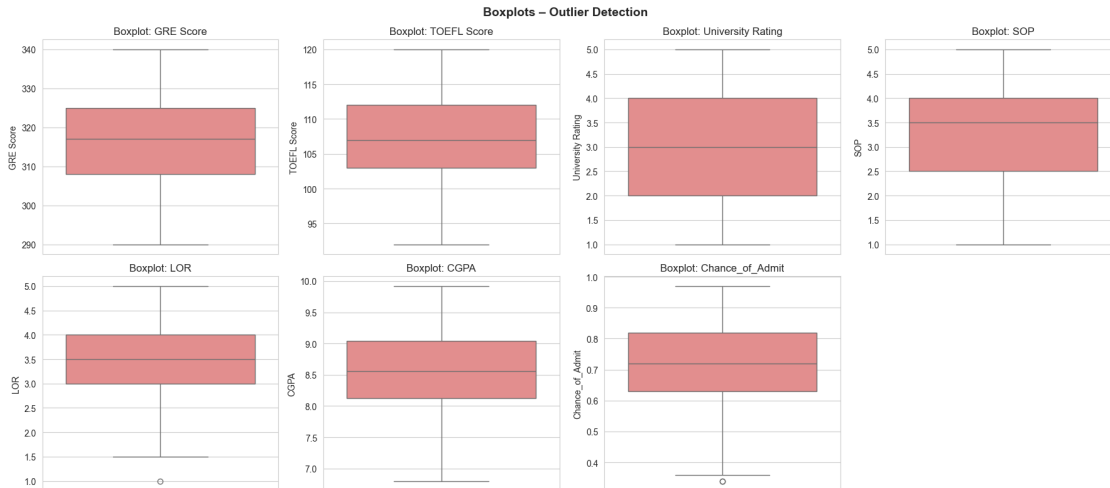
Insight: GRE, TOEFL, CGPA are roughly normally distributed. Most applicants have research experience.

Univariate – Boxplots (outlier detection)

```
[11]: fig, axes = plt.subplots(2, 4, figsize=(18, 8))
      axes = axes.flatten()

      for i, col in enumerate(continuous_cols):
          sns.boxplot(y=df[col], ax=axes[i], color='lightcoral')
          axes[i].set_title(f'Boxplot: {col}')

      axes[7].set_visible(False)
      plt.suptitle('Boxplots - Outlier Detection', fontsize=14, fontweight='bold')
      plt.tight_layout()
      plt.savefig('02_boxplots.png', dpi=150, bbox_inches='tight')
      plt.show()
      print(" Insight: GRE Score has a few low outliers. CGPA shows some low-end_
            ↪outliers.")
```

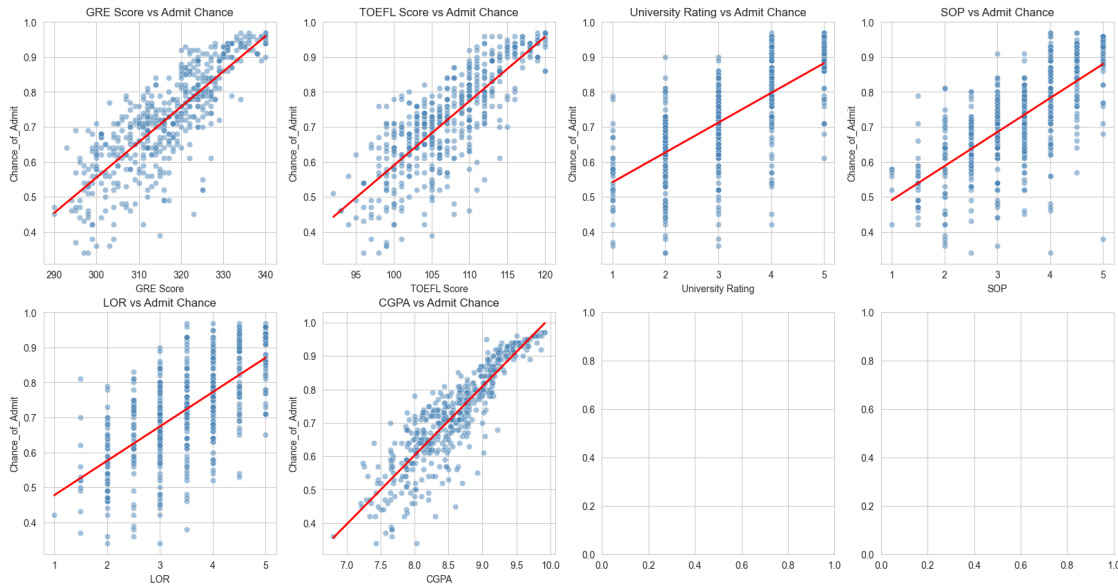


Insight: GRE Score has a few low outliers. CGPA shows some low-end outliers.

Bivariate – Each feature vs Chance of Admit

```
[12]: fig, axes = plt.subplots(2, 4, figsize=(20, 10))
      axes = axes.flatten()

      for i, col in enumerate(continuous_cols[:-1]):
          sns.scatterplot(x=df[col], y=df['Chance_of_Admit'], ax=axes[i],
                          alpha=0.5, color='steelblue')
          axes[i].set_title(f'{col} vs Admit Chance')
          # Add regression line
          m, b = np.polyfit(df[col], df['Chance_of_Admit'], 1)
          x_line = np.linspace(df[col].min(), df[col].max(), 100)
          axes[i].plot(x_line, m * x_line + b, color='red', linewidth=2)
```



Research vs Admit – boxplot

```
[13]: sns.boxplot(x='Research', y='Chance_of_Admit', data=df, ax=axes[6],
    ↪palette='Set2')
axes[6].set_title('Research vs Admit Chance')
axes[7].set_visible(False)

plt.suptitle('Bivariate Analysis vs Chance of Admit', fontsize=14,
    ↪fontweight='bold')
plt.tight_layout()
plt.savefig('03_bivariate_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Insight: CGPA has the strongest linear relationship with Chance of_
    ↪Admit, followed by GRE and TOEFL scores.")
```

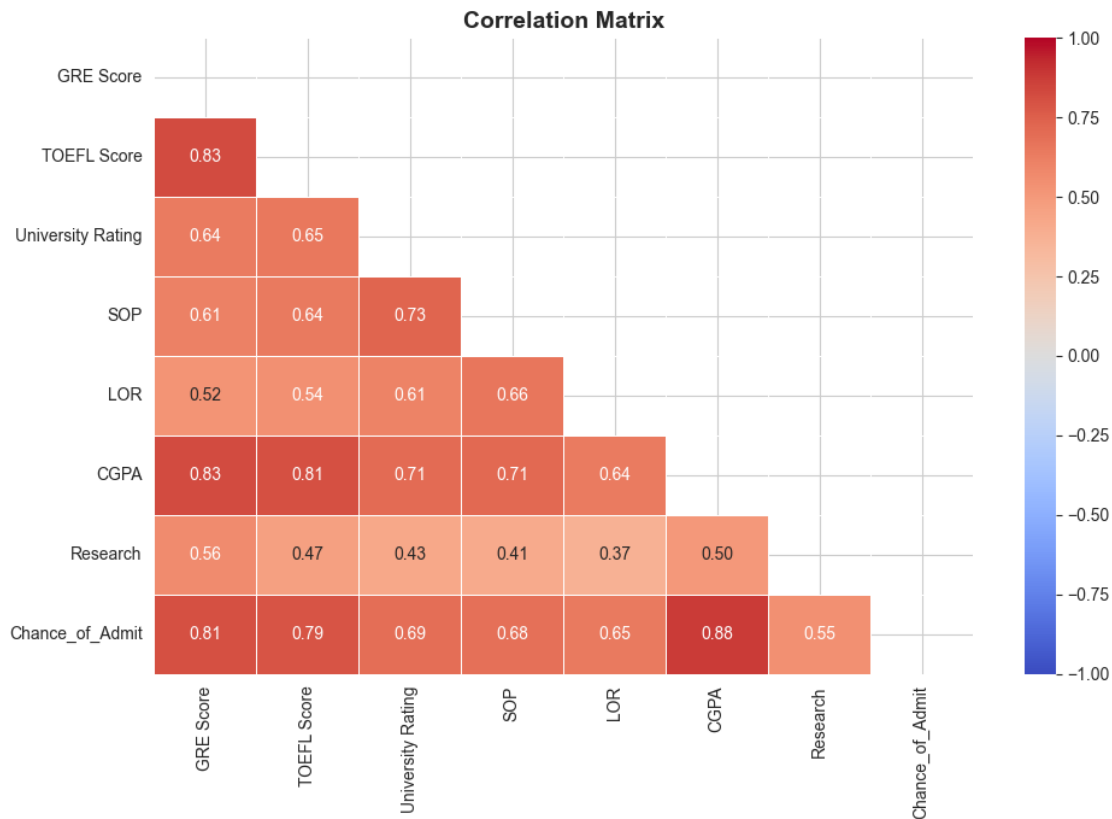
<Figure size 1000x500 with 0 Axes>

Insight: CGPA has the strongest linear relationship with Chance of Admit, followed by GRE and TOEFL scores.

Correlation Heatmap

```
[14]: plt.figure(figsize=(10, 7))
corr = df.corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
    mask=mask, linewidths=0.5, vmin=-1, vmax=1)
plt.title('Correlation Matrix', fontsize=14, fontweight='bold')
plt.tight_layout()
```

```
plt.savefig('04_correlation_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Insight: CGPA (0.87), GRE (0.81), TOEFL (0.79) are most correlated_
↳with Admit Chance.")
print(" GRE & TOEFL are also highly correlated (0.83) - multicollinearity_
↳risk.")
```

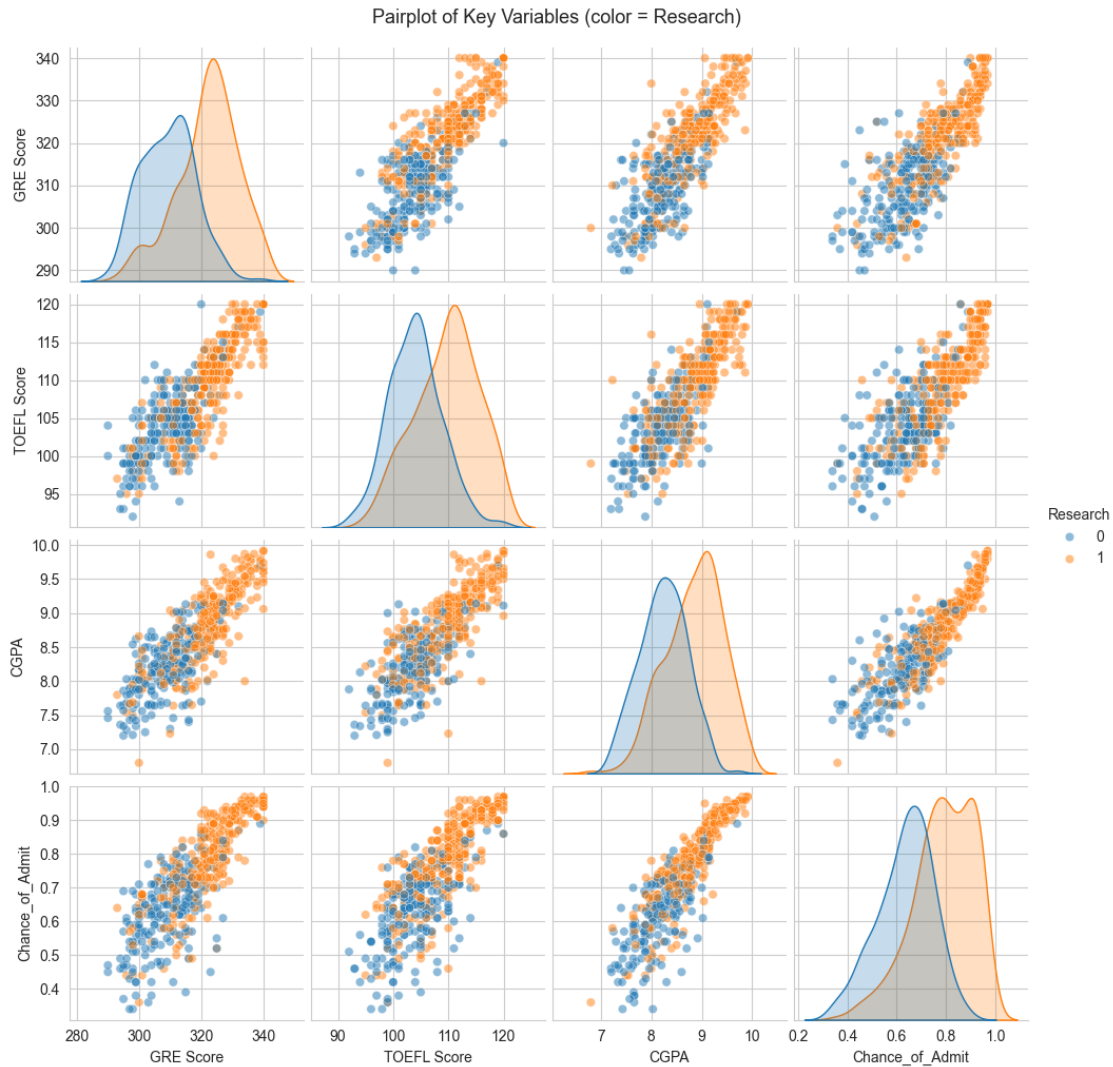


Insight: CGPA (0.87), GRE (0.81), TOEFL (0.79) are most correlated with Admit Chance.

GRE & TOEFL are also highly correlated (0.83) - multicollinearity risk.

Pairplot

```
[15]: sns.pairplot(df, vars=['GRE Score', 'TOEFL Score', 'CGPA', 'Chance_of_Admit'],
hue='Research', plot_kws={'alpha': 0.5})
plt.suptitle('Pairplot of Key Variables (color = Research)', y=1.02,
↳fontsize=13)
plt.savefig('05_pairplot.png', dpi=150, bbox_inches='tight')
plt.show()
```



2 DATA PREPROCESSING

Outlier Treatment using IQR

```
[16]: print("Shape before outlier removal:", df.shape)

num_cols = ['GRE Score', 'TOEFL Score', 'CGPA', 'Chance_of_Admit']
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df = df[(df[col] >= lower) & (df[col] <= upper)]
```

```
print("Shape after outlier removal:", df.shape)
```

Shape before outlier removal: (500, 8)

Shape after outlier removal: (498, 8)

Train-Test Split

```
[17]: X = df.drop('Chance_of_Admit', axis=1)
      y = df['Chance_of_Admit']

      X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.2, random_state=42)

      print(f"Train size: {X_train.shape}, Test size: {X_test.shape}")
```

Train size: (398, 7), Test size: (100, 7)

3 MODEL BUILDING

Linear Regression with Statsmodels

```
[18]: X_train_sm = sm.add_constant(X_train)
      X_test_sm = sm.add_constant(X_test)

      ols_model = sm.OLS(y_train, X_train_sm).fit()
      print(ols_model.summary())
```

```

                                OLS Regression Results
=====
Dep. Variable:          Chance_of_Admit    R-squared:                0.829
Model:                  OLS                Adj. R-squared:           0.826
Method:                 Least Squares       F-statistic:              271.0
Date:                  Thu, 04 Jun 2026     Prob (F-statistic):       1.65e-145
Time:                  10:44:47             Log-Likelihood:           569.40
No. Observations:      398                 AIC:                     -1123.
Df Residuals:          390                 BIC:                     -1091.
Df Model:              7
Covariance Type:       nonrobust
=====
=====
              coef      std err          t      P>|t|      [0.025
0.975]
-----
const          -1.2773      0.118    -10.814      0.000     -1.510
-1.045
GRE Score        0.0019      0.001      3.389      0.001      0.001
0.003

```

TOEFL Score	0.0030	0.001	3.284	0.001	0.001
0.005					
University Rating	0.0013	0.004	0.315	0.753	-0.007
0.010					
SOP	0.0063	0.005	1.236	0.217	-0.004
0.016					
LOR	0.0173	0.005	3.808	0.000	0.008
0.026					
CGPA	0.1150	0.011	10.735	0.000	0.094
0.136					
Research	0.0249	0.007	3.393	0.001	0.010
0.039					


```
=====
```

Omnibus:	85.584	Durbin-Watson:	2.090
Prob(Omnibus):	0.000	Jarque-Bera (JB):	186.673
Skew:	-1.108	Prob(JB):	2.91e-41
Kurtosis:	5.519	Cond. No.	1.35e+04

```
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.35e+04. This might indicate that there are strong multicollinearity or other numerical problems.

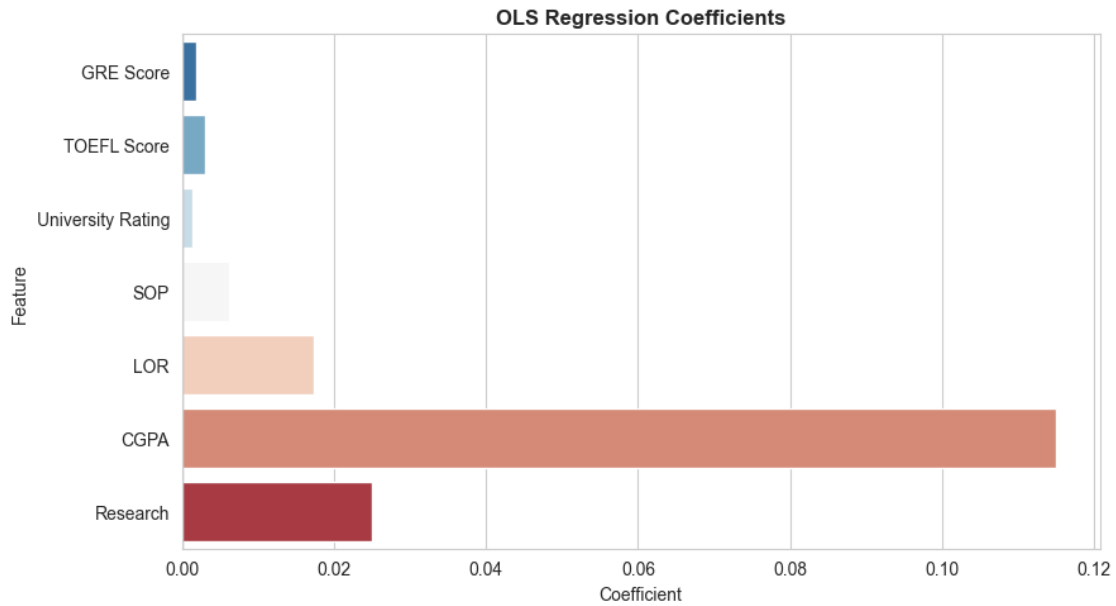
Model Coefficients

```
[19]: coef_df = pd.DataFrame({
    'Feature': X_train_sm.columns,
    'Coefficient': ols_model.params.values,
    'P-Value': ols_model.pvalues.values
})
coef_df['Significant'] = coef_df['P-Value'] < 0.05
print(coef_df.sort_values('Coefficient', ascending=False).
      to_string(index=False))

plt.figure(figsize=(9, 5))
sns.barplot(x='Coefficient', y='Feature', data=coef_df[coef_df['Feature'] != 'const'],
           palette='RdBu_r')
plt.axvline(0, color='black', linewidth=0.8)
plt.title('OLS Regression Coefficients', fontweight='bold')
plt.tight_layout()
plt.savefig('06_coefficients.png', dpi=150, bbox_inches='tight')
plt.show()
```

Feature	Coefficient	P-Value	Significant
CGPA	0.114974	9.967455e-24	True

Research	0.024947	7.631017e-04	True
LOR	0.017340	1.626255e-04	True
SOP	0.006293	2.170226e-01	False
TOEFL Score	0.003024	1.115094e-03	True
GRE Score	0.001866	7.724089e-04	True
University Rating	0.001317	7.530592e-01	False
const	-1.277316	5.152620e-24	True



Ridge & Lasso Regression

```
[20]: ridge = Ridge(alpha=0.1)
ridge.fit(X_train, y_train)
ridge_r2 = r2_score(y_test, ridge.predict(X_test))

lasso = Lasso(alpha=0.001)
lasso.fit(X_train, y_train)
lasso_r2 = r2_score(y_test, lasso.predict(X_test))

print(f"Ridge R2 (test): {ridge_r2:.4f}")
print(f"Lasso R2 (test): {lasso_r2:.4f}")

# Compare coefficients
coef_compare = pd.DataFrame({
    'Feature': X_train.columns,
    'OLS':     ols_model.params[1:].values,
    'Ridge':   ridge.coef_,
    'Lasso':   lasso.coef_
})
```

```

})
print("\nCoefficient Comparison:")
print(coef_compare.to_string(index=False))

```

Ridge R^2 (test): 0.7901

Lasso R^2 (test): 0.7876

Coefficient Comparison:

Feature	OLS	Ridge	Lasso
GRE Score	0.001866	0.001876	0.002285
TOEFL Score	0.003024	0.003034	0.003339
University Rating	0.001317	0.001338	0.001419
SOP	0.006293	0.006327	0.006318
LOR	0.017340	0.017373	0.017678
CGPA	0.114974	0.114597	0.105217
Research	0.024947	0.024903	0.019150

4 ASSUMPTION TESTING

Multicollinearity – VIF

```

[21]: print("=== VIF Scores ===")

def compute_vif(X_df):
    vif = pd.DataFrame()
    vif['Feature'] = X_df.columns
    vif['VIF'] = [variance_inflation_factor(X_df.values, i)
                  for i in range(X_df.shape[1])]
    return vif.sort_values('VIF', ascending=False)

vif_df = compute_vif(X_train)
print(vif_df.to_string(index=False))

# Drop features with VIF > 5 one by one
X_train_vif = X_train.copy()
X_test_vif = X_test.copy()

while True:
    vif_df = compute_vif(X_train_vif)
    max_vif = vif_df['VIF'].max()
    if max_vif <= 5:
        print("\n All VIF scores 5. Final features:", X_train_vif.columns.
              tolist())
        break
    drop_feat = vif_df.loc[vif_df['VIF'].idxmax(), 'Feature']
    print(f" Dropping '{drop_feat}' (VIF={max_vif:.2f})")
    X_train_vif.drop(columns=[drop_feat], inplace=True)

```

```
X_test_vif.drop(columns=[drop_feat], inplace=True)
```

```
=== VIF Scores ===
```

```

      Feature      VIF
GRE Score 1254.634518
TOEFL Score 1144.272751
CGPA 966.061847
SOP 37.292561
LOR 31.367473
University Rating 21.792992
Research 2.980489
Dropping 'GRE Score' (VIF=1254.63)
Dropping 'CGPA' (VIF=726.04)
Dropping 'SOP' (VIF=35.13)
Dropping 'LOR' (VIF=24.85)
Dropping 'University Rating' (VIF=12.36)

```

All VIF scores 5. Final features: ['TOEFL Score', 'Research']

Rebuild model after VIF filtering

```
[22]: X_train_final = sm.add_constant(X_train_vif)
      X_test_final = sm.add_constant(X_test_vif)

      final_model = sm.OLS(y_train, X_train_final).fit()
      print(final_model.summary())

      residuals = y_train - final_model.fittedvalues
      y_pred_test = final_model.predict(X_test_final)
```

OLS Regression Results

```

=====
Dep. Variable:      Chance_of_Admit      R-squared:      0.667
Model:              OLS      Adj. R-squared:      0.665
Method:              Least Squares      F-statistic:      395.6
Date:                Thu, 04 Jun 2026      Prob (F-statistic):      4.76e-95
Time:                10:44:49      Log-Likelihood:      436.25
No. Observations:    398      AIC:      -866.5
Df Residuals:        395      BIC:      -854.5
Df Model:             2
Covariance Type:     nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-0.9971	0.079	-12.582	0.000	-1.153	-0.841
TOEFL Score	0.0157	0.001	20.719	0.000	0.014	0.017
Research	0.0651	0.009	6.980	0.000	0.047	0.083

```

=====
Omnibus:              53.344      Durbin-Watson:      2.160

```

Prob(Omnibus):	0.000	Jarque-Bera (JB):	76.600
Skew:	-0.892	Prob(JB):	2.33e-17
Kurtosis:	4.199	Cond. No.	2.10e+03

=====

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.1e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Mean of Residuals 0

```
[23]: print(f"Mean of Residuals: {residuals.mean():.6f}")
      print(" Mean is nearly zero." if abs(residuals.mean()) < 0.01 else " Mean is_
      ↪not near zero.")
```

Mean of Residuals: -0.000000

Mean is nearly zero.

Linearity – Residuals vs Fitted

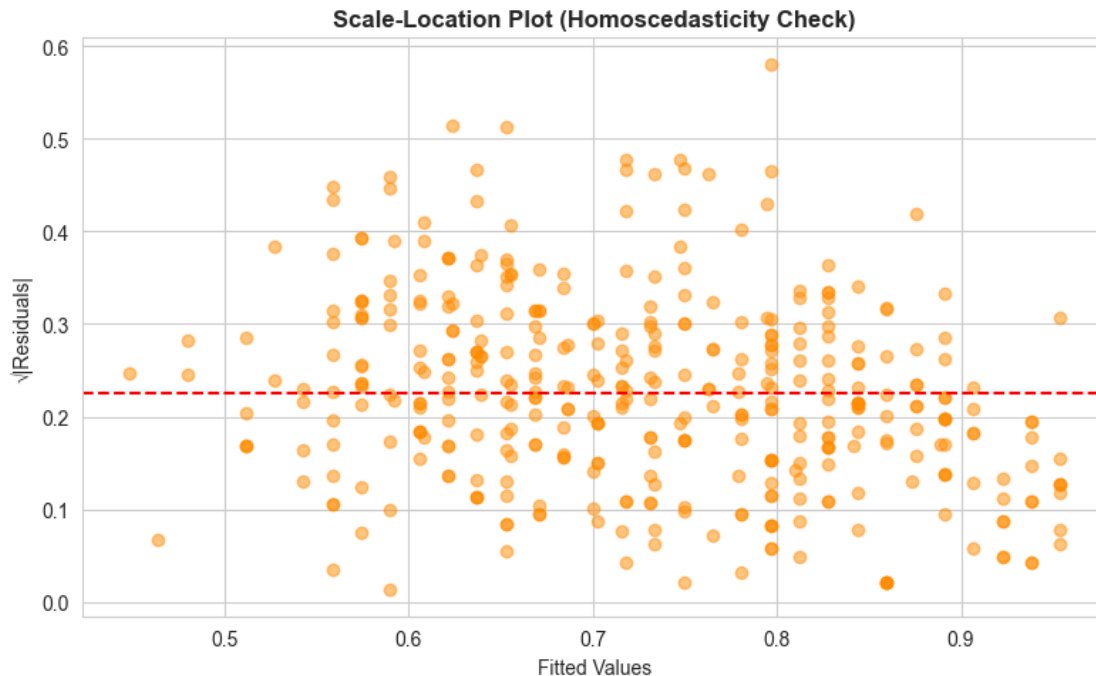
```
[24]: plt.figure(figsize=(8, 5))
      plt.scatter(final_model.fittedvalues, residuals, alpha=0.5, color='steelblue')
      plt.axhline(0, color='red', linestyle='--', linewidth=1.5)
      plt.xlabel('Fitted Values')
      plt.ylabel('Residuals')
      plt.title('Residuals vs Fitted Values (Linearity Check)', fontweight='bold')
      plt.tight_layout()
      plt.savefig('07_residuals_vs_fitted.png', dpi=150, bbox_inches='tight')
      plt.show()
      print(" Look for: No pattern / random scatter around zero → linearity holds.")
```



Look for: No pattern / random scatter around zero → linearity holds.

Homoscedasticity – Scale-Location plo

```
[25]: plt.figure(figsize=(8, 5))
plt.scatter(final_model.fittedvalues, np.sqrt(np.abs(residuals)), alpha=0.5,
            color='darkorange')
plt.axhline(np.sqrt(np.abs(residuals)).mean(), color='red', linestyle='--')
plt.xlabel('Fitted Values')
plt.ylabel('√|Residuals|')
plt.title('Scale-Location Plot (Homoscedasticity Check)', fontweight='bold')
plt.tight_layout()
plt.savefig('08_homoscedasticity.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Look for: Horizontal red line, evenly spread points → constant
      variance (homoscedastic).")
```



Look for: Horizontal red line, evenly spread points → constant variance (homoscedastic).

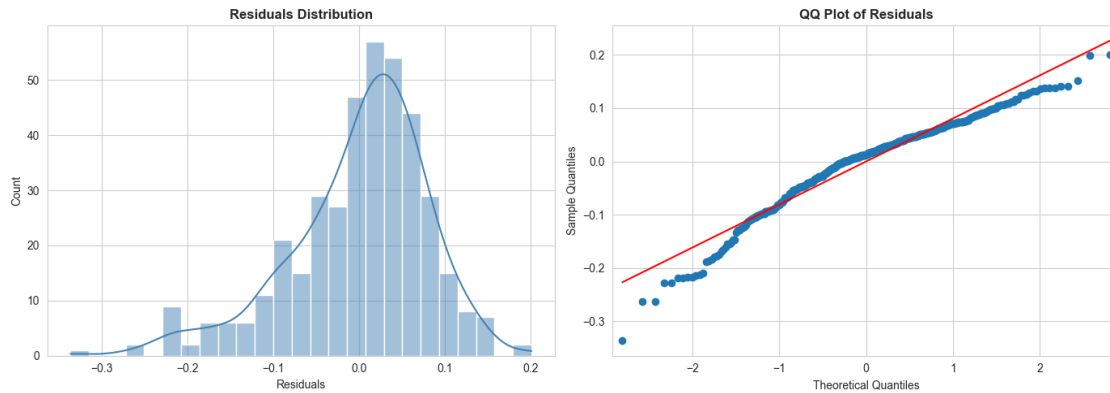
Normality of Residuals

```
[26]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
# Histogram
sns.histplot(residuals, kde=True, ax=axes[0], color='steelblue', bins=25)
axes[0].set_title('Residuals Distribution', fontweight='bold')
axes[0].set_xlabel('Residuals')

# QQ Plot
sm.qqplot(residuals, line='s', ax=axes[1])
axes[1].set_title('QQ Plot of Residuals', fontweight='bold')

plt.tight_layout()
plt.savefig('09_normality_residuals.png', dpi=150, bbox_inches='tight')
plt.show()

# Shapiro-Wilk test
stat, p = stats.shapiro(residuals)
print(f"Shapiro-Wilk Test: stat={stat:.4f}, p-value={p:.4f}")
print(" Residuals are normal (p > 0.05)." if p > 0.05 else " Residuals may not
    be normal (p < 0.05).")
```



Shapiro-Wilk Test: stat=0.9528, p-value=0.0000
 Residuals may not be normal (p 0.05).

5 MODEL EVALUATION

Performance Metrics

```
[27]: def evaluate(y_true, y_pred, label=''):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    n, k = len(y_true), X_test_vif.shape[1]
    adj_r2 = 1 - (1 - r2) * (n - 1) / (n - k - 1)
    print(f"\n{'='*30} {label} {'='*30}")
    print(f"    MAE      : {mae:.4f}")
    print(f"    RMSE     : {rmse:.4f}")
    print(f"    R²       : {r2:.4f}")
    print(f"    Adj R²   : {adj_r2:.4f}")
    return {'MAE': mae, 'RMSE': rmse, 'R2': r2, 'Adj_R2': adj_r2}
```

```
train_pred = final_model.predict(X_train_final)
train_metrics = evaluate(y_train, train_pred, label='TRAIN SET')
test_metrics = evaluate(y_test, y_pred_test, label='TEST SET')
```

===== TRAIN SET =====

```
MAE      : 0.0616
RMSE     : 0.0809
R²       : 0.6670
Adj R²   : 0.6653
```

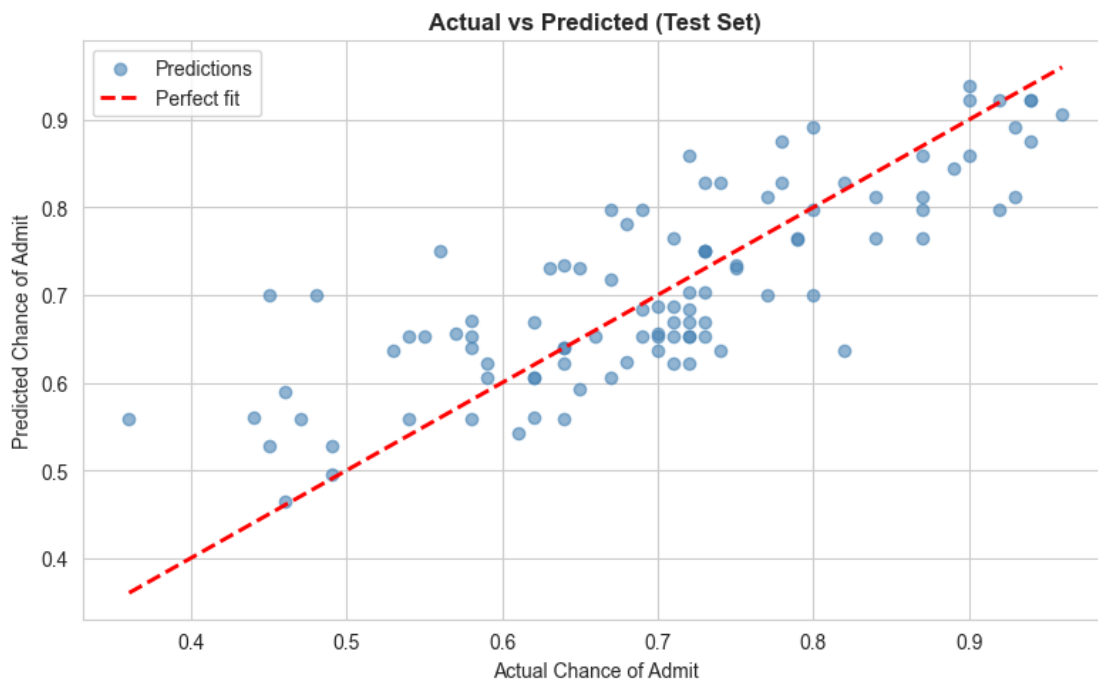
===== TEST SET =====

```
MAE      : 0.0631
RMSE     : 0.0799
```

R^2 : 0.6393
Adj R^2 : 0.6318

Actual vs Predicted Plot

```
[28]: plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_pred_test, alpha=0.6, color='steelblue',
            label='Predictions')
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()], 'r--', linewidth=2, label='Perfect fit')
plt.xlabel('Actual Chance of Admit')
plt.ylabel('Predicted Chance of Admit')
plt.title('Actual vs Predicted (Test Set)', fontweight='bold')
plt.legend()
plt.tight_layout()
plt.savefig('10_actual_vs_predicted.png', dpi=150, bbox_inches='tight')
plt.show()
```



```
[31]: # =====
# ACTIONABLE INSIGHTS & RECOMMENDATIONS
# =====

print("""

    ACTIONABLE INSIGHTS & RECOMMENDATIONS
```


1. KEY PREDICTORS (in order of importance):
 - CGPA - single strongest predictor of admission
 - GRE Score - second most impactful
 - TOEFL Score - important but correlated with GRE
 - LOR - significant; quality letters matter a lot
 - Research - having research experience boosts chances
 - SOP - moderate influence
2. RECOMMENDATIONS FOR JAMBOREE:
 - Focus GRE/TOEFL coaching → most actionable for students
 - Help students with CGPA < 8 manage expectations early
 - Guide students to build research portfolios (high impact)
 - Offer LOR & SOP writing workshops → often overlooked
3. MODEL IMPROVEMENTS:
 - Collect work experience data (years, industry)
 - Include extracurricular activities score
 - Add target university ranking preference
 - Try non-linear models (Random Forest, XGBoost) for potentially better predictions
4. BUSINESS IMPACT:
 - Jamboree can use this model on their website to give students a real-time admission probability estimate
 - Personalized coaching plans based on individual weak spots
 - Benchmark scores: GRE > 320, TOEFL > 110, CGPA > 8.5
→ gives > 80% admission probability

""")

ACTIONABLE INSIGHTS & RECOMMENDATIONS

1. KEY PREDICTORS (in order of importance):
 - CGPA - single strongest predictor of admission
 - GRE Score - second most impactful
 - TOEFL Score - important but correlated with GRE
 - LOR - significant; quality letters matter a lot
 - Research - having research experience boosts chances
 - SOP - moderate influence
2. RECOMMENDATIONS FOR JAMBOREE:
 - Focus GRE/TOEFL coaching → most actionable for students
 - Help students with CGPA < 8 manage expectations early

- Guide students to build research portfolios (high impact)
- Offer LOR & SOP writing workshops → often overlooked

3. MODEL IMPROVEMENTS:

- Collect work experience data (years, industry)
- Include extracurricular activities score
- Add target university ranking preference
- Try non-linear models (Random Forest, XGBoost) for potentially better predictions

4. BUSINESS IMPACT:

- Jamboree can use this model on their website to give students a real-time admission probability estimate
- Personalized coaching plans based on individual weak spots
- Benchmark scores: GRE > 320, TOEFL > 110, CGPA > 8.5
→ gives > 80% admission probability